



Tập 1: Network Model & Pod

Lý thuyết: 4 Nguyên tắc không NAT, Pause Container & veth pair

Network Thực Chiến · Series: Kubernetes Networking · Tập 01

Nội dung

1. **Kubernetes Network Model** — Tờ kế ước 4 nguyên tắc bất biến
2. **Pause Container** — "Cái giá đỡ mạng" ẩn mình trong mọi Pod
3. **veth pair** — Cầu nối giữa Pod và Node
4. **Kiểm tra thực tế** — `ip netns`, `nsenter`, `crictl`

Phần 1

Kubernetes Network Model

Tờ kế ước nền tảng

K8s không quy định bạn phải dùng công nghệ mạng nào. Nhưng nó đặt ra **4 nguyên tắc bất biến** mà bất kỳ CNI plugin nào cũng phải tuân thủ:

1. **Pod-to-Pod không NAT**: Mọi Pod đều có thể giao tiếp với mọi Pod khác trên bất kỳ Node nào mà **không cần NAT**.
2. **Node-to-Pod không NAT**: Node có thể liên lạc trực tiếp với bất kỳ Pod nào mà không cần NAT.
3. **IP của Pod là cố định**: IP mà Pod tự "nhìn thấy" bản thân (`eth0` bên trong Pod) phải là **đúng IP** mà các Pod khác dùng để liên lạc với nó.
4. **Mọi Pod có IP riêng**: Không có chuyện 2 Pod chia sẻ cùng 1 IP.

Flat Network Model

Node 1 (192.168.1.10)

Pod A: 10.244.0.5
Pod B: 10.244.0.6

Node 2 (192.168.1.11)

Pod C: 10.244.1.3
Pod D: 10.244.1.4

____ CNI Plugin ____
(Flannel / Calico / Cilium)

Pod A (10.244.0.5) → Pod C (10.244.1.3)

- ✓ Đi thẳng – KHÔNG qua NAT
- ✓ Source IP giữ nguyên là 10.244.0.5

Đây là lý do mọi CNI plugin đều implement Overlay (VXLAN/IPIP) hoặc BGP routing — để đảm bảo tính no-NAT.

Tại sao no-NAT quan trọng?

Scenario	Với NAT	Không NAT (K8s model)
App log source IP	✗ Log thấy IP Node, không phải Pod	✓ Log thấy IP Pod thực
Distributed tracing	✗ Trace bị đứt đoạn tại NAT	✓ Trace xuyên suốt
Firewall rule	✗ Phải rule cho từng Node IP	✓ Rule theo Pod IP/subnet
Service discovery	✗ Cần thêm layer discovery	✓ DNS → Pod IP thẳng

Phần 2

Pause Container

Pause Container là gì?

Mỗi Pod trong K8s thực chất chứa **ít nhất 2 containers**:

```
Pod: my-app
├── pause container (infra container)  ← Ẩn, không thấy trong kubectl
│   ├── Giữ Network Namespace
│   ├── Giữ IPC Namespace
│   └── PID = 1 (tránh zombie process)
└── my-app container                  ← Container bạn tạo ra
    └── JOIN vào Network NS của pause
```

`pause` container image siêu nhỏ (~700KB), chỉ chạy 1 syscall: `pause()`.

Nó là "**cái giá đỡ**" giữ Network Namespace tồn tại ngay cả khi app container restart.

Tại sao cần Pause Container?

Vấn đề: Nếu không có pause, Network NS sẽ biến mất khi container restart.

Không có pause:

App crash → Container bị xóa → Network NS biến mất
→ Pod IP thay đổi → Service discovery sai!

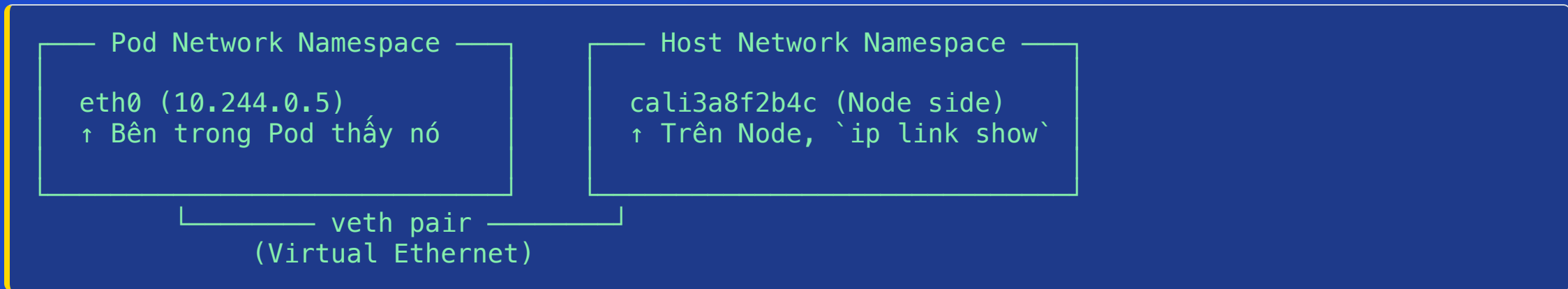
Có pause:

App crash → Container restart (pause vẫn sống)
→ Network NS GIỮ NGUYÊN → Pod IP KHÔNG ĐỔI ✓

Kiểm tra pause container đang chạy:

```
# Trên worker node (cần ssh vào)
sudo crictl ps | grep pause
# CONTAINER    IMAGE      CREATED    STATE    NAME    POD ID
# abc123       pause     5m        Running  pause  xyz789...
```

veth pair: Cầu nối Pod ↔ Node



veth pair hoạt động như dây cáp ảo:

- Một đầu trong Pod (gọi là `eth0`)
- Một đầu ngoài Node (gọi là `cali...`, `veth...`)
- Packet vào một đầu → **ngay lập tức xuất hiện** ở đầu kia

⚠️ "Namespace" — 2 khái niệm hoàn toàn khác nhau

Cùng 1 từ, 2 tầng kỹ thuật khác nhau — đây là nguồn gốc nhầm lẫn phổ biến nhất:

	Kubernetes Namespace	Linux Network Namespace
Là gì	Phân vùng logic cho K8s resources	Cô lập kernel: interfaces, routing table, iptables
Lệnh	<code>kubectl get ns</code> · <code>kubectl -n prod</code>	<code>ip netns list</code> · <code>ip netns exec</code>
Tạo bởi	Admin/Dev (<code>kubectl create namespace</code>)	CNI plugin (tự động khi Pod được schedule)
Cô lập mạng?	❌ Không — chỉ cô lập resource view	✅ Có — Pod có riêng eth0, routing table, iptables

K8s Namespace "production":

`kubectl -n production get pods`

↑ Gom nhóm resources cho dễ quản lý

↑ Pod khác namespace vẫn ping được

Linux Network Namespace (của Pod):

`/var/run/netns/cni-abc12345-...`

↑ Cô lập kernel thật sự

↑ Pod có ip riêng, route riêng, iptables riêng

Quy tắc nhớ: Nói về **Pod networking** (IP, route, interface) → luôn là **Linux Network Namespace**.

Nói về `kubectl -n`, phân quyền, resource isolation → luôn là **Kubernetes Namespace**.

Phần 3

Kiểm tra thực tế

ip netns: Xem Network Namespaces

```
# Trên worker node – liệt kê tất cả network namespaces
sudo ip netns list
# cni-abc12345-1234-5678-abcd-123456789012 (id: 5)
# cni-def67890-...

# Chạy lệnh trong một namespace cụ thể
sudo ip netns exec cni-abc12345-... ip addr show
# 1: lo: <LOOPBACK,UP> ...
# 2: eth0@if7: <BROADCAST,UP> mtu 1450 ...
#    inet 10.244.0.5/32 scope global eth0

# Xem veth pair: số sau "@if" là index của đầu bên Node
sudo ip link show | grep "^7:"
# 7: cali3a8f2b4c@if2: <BROADCAST,UP> ...
```

nsenter: Chui vào Pod Network NS

```
# Lấy PID của pause container của Pod
POD="my-app-xxx"
PAUSE_ID=$(kubectl get pod $POD \
  -o jsonpath='{.status.containerStatuses[0].containerID}' \
  | sed 's/containerd:\/\///')
PID=$(sudo crictl inspect $PAUSE_ID \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['info']['pid'])")

# Chui vào network namespace, dùng tools từ Node
sudo nsenter -t $PID -n ip addr show
sudo nsenter -t $PID -n ip route show
sudo nsenter -t $PID -n ss -tlnp
```

Hữu ích khi cần debug Pod không có tools (`distroless` image).

Key Takeaways

Khái niệm	Tóm tắt
4 nguyên tắc K8s	No-NAT Pod-to-Pod, no-NAT Node-to-Pod, IP cố định, IP riêng
Pause container	Giữ Network NS ngay cả khi app container restart
veth pair	"Dây cáp ảo" nối Pod NS với Host NS
K8s Namespace	Phân vùng logic resources — <code>kubectl -n</code> — KHÔNG cô lập mạng
Linux Network Namespace	Cô lập kernel thật sự — <code>ip netns</code> — tạo ra mạng riêng cho Pod
nsenter	Dùng tools của Node để debug network trong Pod

Cảm ơn đã theo dõi! 🙏

Network Thực Chiến

Tập tiếp theo: **Tập 2 — CNI Specification: Cơ chế plugin hoạt động**

"K8s không quan tâm bạn dùng mạng gì — miễn là không có NAT."